

1. Введение

Настоящий документ представляет собой сводный отчёт по лабораторной работе № 4 «Технологии промышленной разработки программного обеспечения. Сертификация» для сертифицируемого программного продукта «веб-сервис для подбора кандидатов».

Отчёт объединяет:

- техническое задание на сертифицируемый продукт;
- проектную модель уровня платформы, разработанную в предыдущем семестре;
- план квалификационного тестирования с тест-кейсами;
- оценки полноты плана тестирования (по составу тестов и по продолжительности);
- оценки адекватности и соответствия среды тестирования;
- оценку качества документирования тестов и их воспроизводимости;
- оценку достаточности предполагаемой автоматизации тестирования;
- мотивированное экспертное заключение о соответствии предъявленных материалов требованиям сертификации.

Документ предназначен для использования при имитации процедуры сертификации и для обоснования качества разработанного веб-сервиса в учебной среде.

2. Техническое задание на сертифицируемый программный продукт

Техническое задание на сертифицируемый программный продукт

1. Общие сведения

Наименование продукта: веб-сервис для подбора кандидатов.

Тип продукта: распределённое клиент-серверное веб-приложение с микросервисной архитектурой.

Назначение документа: зафиксировать требования к сертифицируемому

программному продукту, выступающему объектом квалификационного тестирования и последующей сертификации.

Продукт представляет собой информационную систему, поддерживающую процессы подбора персонала на основе обработки резюме, управления вакансиями и вычисления степени соответствия профилей кандидатов требованиям работодателей. Система реализуется как набор взаимодействующих микросервисов с использованием современных технологий веб-разработки, обработки естественного языка и контейнеризации.

2. Основания для разработки

Разработка ведётся в рамках учебного проекта по дисциплине «Технологии промышленной разработки программного обеспечения» и базируется на:

- результатах лабораторных работ по требованиям и проектированию (концептуальная модель, диаграммы прецедентов, классов и взаимодействий);
- архитектурной модели уровня платформы (микросервисная архитектура, Docker, Kubernetes, PostgreSQL);
- выбранной предметной областью — автоматизация процесса подбора кандидатов.

Настоящее техническое задание определяет состав и качество функций продукта, а также требования к среде эксплуатации и к его сертификационному испытанию.

3. Назначение и область применения продукта

Веб-сервис предназначен для:

- автоматизации обработки входящих резюме кандидатов;
- ведения каталога вакансий и требований к ним;
- вычисления степени соответствия кандидатов конкретным вакансиям;
- предоставления пользователям (кандидатам, HR-специалистам, администраторам) удобного интерфейса взаимодействия с системой.

Продукт может применяться:

- в отделах подбора персонала компаний малого и среднего бизнеса;
- в рекрутинговых агентствах;
- в учебных лабораторных стендах как демонстрационный пример промышленной архитектуры и процессов сертификации ПО.

4. Акторы и сценарии использования

4.1 Акторы

- **Кандидат** — физическое лицо, подающее резюме и откликающееся на вакансии.
- **HR-специалист (работодатель)** — пользователь, создающий и сопровождающий вакансии, анализирующий рекомендации по кандидатам.
- **Администратор системы** — пользователь с расширенными правами управления пользователями и конфигурацией.

4.2 Основные сценарии

1. **Регистрация и аутентификация пользователей**
2. Регистрация кандидата.
3. Регистрация HR-специалиста.
4. Вход в систему и выход из неё.
5. Восстановление пароля и подтверждение учётной записи.
6. **Работа кандидата**
7. Загрузка и обновление резюме.
8. Просмотр списка рекомендованных вакансий.
9. Подача откликов на выбранные вакансии.
10. **Работа HR-специалиста**
11. Создание, редактирование, публикация и деактивация вакансий.
12. Получение ранжированного списка кандидатов под выбранную вакансию.
13. Просмотр профилей кандидатов и историй откликов.
14. Фиксация результатов рассмотрения (принят/отклонён, комментарии).
15. **Администрирование**
16. Управление ролями и доступом пользователей.
17. Просмотр агрегированных сведений о работе сервиса (логические показатели, ошибки).

5. Требования к функциональности

5.1 Модуль управления пользователями (User/Auth Service)

Система должна:

- обеспечивать регистрацию пользователей с ролями «кандидат», «HR-специалист», «администратор»;
- выполнять аутентификацию по логину/паролю с использованием безопасного хранения паролей (хэширование);
- поддерживать выдачу и проверку токенов доступа (например, JWT);

- реализовывать механизм восстановления пароля (через электронную почту или альтернативный подтверждающий канал);
- предоставлять интерфейсы для изменения профиля пользователя (контактные данные, базовая информация).

Доступ к функциям других модулей должен контролироваться на основе ролей и прав доступа.

5.2 Модуль загрузки и анализа резюме (ResumeService + MLAnalyzer)

Система должна:

- принимать резюме в распространённых форматах (не менее PDF и DOCX);
- выполнять предобработку текста резюме (извлечение текста, очистка служебных символов);
- сохранять структурированное представление резюме (образование, опыт, навыки, контактные данные);
- выполнять векторизацию содержимого резюме с использованием методов обработки естественного языка;
- сохранять результаты векторизации для дальнейшего использования модулем сопоставления;
- поддерживать обновление уже загруженных резюме.

При ошибках чтения или неподдерживаемом формате система должна возвращать пользователю информативные сообщения об ошибке без нарушения целостности хранимых данных.

5.3 Модуль управления вакансиями (VacancyService)

Система должна:

- позволять HR-специалисту создавать вакансии с указанием:
- наименования позиции;
- описания обязанностей;
- требований (квалификация, опыт, hard/soft skills);
- условий (график, уровень компенсации и т.п.);
- обеспечивать просмотр, редактирование и архивирование вакансий;
- хранить статус вакансии (активна, приостановлена, закрыта);
- выполнять базовую валидацию данных (наличие обязательных полей, корректные типы значений).

Вакансии должны быть доступны для сопоставления с резюме кандидатов и для отображения в пользовательском интерфейсе.

5.4 Модуль сопоставления кандидатов и вакансий (MatchingService)

Система должна:

- принимать на вход векторные представления вакансий и резюме;
- вычислять меру сходства между вакансиями и резюме (косинусное сходство или эквивалентная метрика) с результатом в диапазоне $[0; 1]$;
- формировать ранжированный список кандидатов для выбранной вакансии по убыванию значения меры соответствия;
- обеспечивать детерминированность результата при одинаковых входных данных;
- корректно обрабатывать граничные случаи (нулевые векторы, отсутствие кандидатов, неодинаковая размерность входов в рамках принятой политики валидации).

Результаты сопоставления должны быть доступны для вывода в интерфейс HR-специалиста и могут использоваться для последующего анализа эффективности подбора.

5.5 Пользовательский интерфейс (Frontend)

Система должна предоставлять веб-интерфейс, обеспечивающий:

- для кандидата:
 - регистрацию, вход и выход;
 - загрузку, просмотр и обновление резюме;
 - просмотр списка вакансий и рекомендованных позиций;
 - подачу откликов на вакансии;
- для HR-специалиста:
 - создание и редактирование вакансий;
 - просмотр рекомендованных кандидатов для выбранной вакансии;
 - просмотр резюме и фиксацию решений по кандидатам;
- для администратора:
 - просмотр списка пользователей;
 - изменение ролей и блокировку учётных записей при необходимости.

Интерфейс должен быть реализован по принципам удобства использования: логичная структура страниц, понятные подписи элементов, корректное отображение сообщений об ошибках.

6. Требования к данным и хранилищу

Система должна использовать реляционную СУБД (PostgreSQL или эквивалент) для хранения:

- учётных записей пользователей;
- структурированных данных резюме;
- описаний вакансий;
- информации об откликах и результатах рассмотрения;
- технических логов и служебных записей (при необходимости).

Требования:

- поддержка транзакционной целостности (ACID) для операций, изменяющих критичные сущности (пользователи, вакансии, отклики);
- использование механизмов индексации для ускорения поиска по основным полям (идентификаторы, статус, дата обновления);
- защита от несанкционированного доступа к персональным данным на уровне СУБД и прикладной логики.

7. Требования к надёжности и отказоустойчивости

Система должна:

- обеспечивать корректное восстановление после сбоев в пределах возможностей программно-аппаратной платформы;
- не допускать потери уже зафиксированных в базе данных сведений при отказе одного из микросервисов (при условии сохранения работоспособности СУБД);
- обрабатывать сетевые ошибки при межсервисном взаимодействии с выдачей контролируемых сообщений об ошибках и возможностью повторной попытки.

При невозможности выполнить операцию система должна сохранять консистентное состояние данных и информировать пользователя о результате.

8. Требования к производительности

В учебной конфигурации система должна обеспечивать:

- обработку типичных пользовательских операций (загрузка резюме, просмотр списка вакансий, получение рекомендаций) с временем отклика, приемлемым для интерактивной работы (ориентировочно до нескольких секунд при нормальной нагрузке);
- возможность масштабирования отдельных микросервисов (в первую очередь модуля анализа резюме и сопоставления) за счёт развёртывания дополнительных экземпляров в контейнерной среде.

Точные количественные показатели производительности в рамках учебного проекта могут задаваться ориентировочно и уточняться в процессе тестирования.

9. Требования к безопасности

Система должна:

- реализовывать аутентификацию всех пользователей;
- разграничивать доступ к операциям и данным на основе ролей (кандидат, HR-специалист, администратор);
- обеспечивать безопасное хранение паролей (хранение только хэшей с использованием стойких алгоритмов);
- защищать персональные данные кандидатов и контактные данные работодателей от несанкционированного доступа;
- использовать защищённый транспортный уровень (HTTPS) в целевой среде эксплуатации.

Журналы доступа и ошибок, если ведутся, не должны содержать открытых паролей и других конфиденциальных данных.

10. Требования к среде функционирования

10.1 Программно-аппаратная платформа

Базовая целевая среда:

- серверная платформа: Ubuntu Server актуальной LTS-версии;
- контейнеризация: Docker;
- оркестрация (для развёртывания в промышленной конфигурации или её учебного аналога): Kubernetes или локальные средства (например, Minikube) при лабораторной эксплуатации;
- СУБД: PostgreSQL;
- сервер приложений/обратный прокси: Nginx или эквивалент.

10.2 Средства разработки и тестирования

Для разработки:

- backend: язык программирования высокого уровня (например, Python с фреймворком Django или аналогичный стек);
- frontend: современный JavaScript-фреймворк (React или эквивалент) с поддержкой SPA;

- система контроля версий: Git.

Для тестирования и автоматизации допускается использование средств unit-тестирования, средств тестирования API и систем непрерывной интеграции.

11. Требования к эргономике и удобству использования

Интерфейс системы должен:

- быть однозначно понятен пользователям целевых ролей без необходимости изучения специальной документации;
- содержать чёткие формулировки сообщений об ошибках и результатах операций;
- обеспечивать консистентный дизайн элементов управления на всех страницах.

Навигация по системе должна соответствовать логике бизнес-процессов кандидата и HR-специалиста: от регистрации и загрузки резюме/создания вакансий к просмотру рекомендаций и фиксации решений.

12. Требования к документированию

К продукту должны быть подготовлены следующие документы:

- пользовательская инструкция (краткое руководство для кандидата и HR-специалиста);
- описание архитектуры и компонентов (для администраторов и разработчиков);
- спецификация тестов, использованных при квалификационном тестировании сертифицируемого продукта.

Документы должны быть оформлены в едином стиле, с нумерацией разделов и ссылками на элементы системы (экраны, модули, API).

13. Требования к сопровождению и развитию

Продукт должен быть спроектирован с возможностью:

- добавления новых типов пользователей и ролей;
- расширения набора поддерживаемых форматов резюме;
- доработки алгоритмов сопоставления (изменение метрик, добавление учёта обратной связи и дополнительных признаков);
- интеграции с внешними сервисами (например, внешние каталоги вакансий или системы учёта кандидатов).

Все изменения должны сопровождаться обновлением документации и, при необходимости, пересмотром плана квалификационного тестирования.

14. Требования, связанные с сертификацией

Продукт должен быть подготовлен к проведению квалификационного тестирования и сертификационных испытаний. Для этого необходимо:

- обеспечить воспроизводимое развёртывание тестовой среды, максимально приближенной к целевой;
- подготовить набор тест-кейсов, покрывающих ключевые функциональные и нефункциональные требования настоящего технического задания;
- зафиксировать результаты испытаний и выводы о соответствии продукта заявленным требованиям.

По итогам испытаний должно быть сформировано мотивированное заключение о соответствии (или несоответствии) программного продукта требованиям технического задания.

3. Проектная модель уровня платформы (по материалам предыдущего семестра)

Архитектурное проектирование веб-сервиса для подбора кандидатов выполнено в предыдущем семестре в рамках лабораторной работы №3 по разделу «Проектирование». Ниже приводятся ключевые элементы проектной модели уровня платформы, необходимые для целей сертификации и тестирования.

3.1 Общая архитектурная концепция

Веб-сервис реализуется как распределённое приложение с микросервисной архитектурой. Основные компоненты:

- сервис загрузки и анализа резюме (ResumeService + MLAnalyzer);
- сервис управления вакансиями (VacancyService);
- сервис сопоставления кандидатов и вакансий (MatchingService);
- сервис управления пользователями и аутентификацией (User/Auth Service);
- фронтенд-приложение для работы кандидатов, HR-специалистов и администраторов.

Компоненты взаимодействуют по REST/HTTP, обмениваясь данными в формате JSON, а хранилищем служит PostgreSQL.

Продукт развёртывается в контейнерах Docker, в промышленной конфигурации предполагается использование Kubernetes для оркестрации.

3.2 Краткая характеристика ключевых сервисов

- **ResumeService.** Приём файлов резюме (PDF/DOCX), валидация, предобработка текста, передача данных в MLAnalyzer, сохранение структурированных профилей кандидатов и векторных представлений.
- **VacancyService.** Создание, редактирование, публикация и архивирование вакансий, хранение требований и условий, предоставление API для сопоставления и пользовательского интерфейса.
- **MatchingService.** Ядро интеллектуальной логики, реализующее методы CalculateSimilarity и RankCandidates для сравнения векторных представлений резюме и вакансий и формирования ранжированных списков кандидатов.
- **User/Auth Service.** Регистрация, аутентификация и авторизация по ролям, управление профилями пользователей, выдача и проверка токенов доступа.
- **Frontend.** SPA-клиент (React/Vue.js), предоставляющий интерфейс для всех ролей пользователей и взаимодействующий с backend через REST API.

3.3 Операционная система и среда проектирования

В качестве базовой операционной системы в эксплуатационной конфигурации выбрана Ubuntu Server LTS, обеспечивающая стабильность, предсказуемую поддержку и высокую совместимость с Docker и Kubernetes.

Для разработки и отладки backend-логики используется связка Python + Django + PostgreSQL (среда разработки PyCharm), для frontend — React + TypeScript (VS Code). ML-модели разрабатываются и валидируются в среде Jupyter Notebook с использованием библиотеки HuggingFace Transformers.

Такая проектная модель уровня платформы соответствует требованиям ТЗ и задаёт контекст для подготовки плана тестирования и организации среды квалификационных испытаний.

4. План тестирования веб-сервиса подбора кандидатов

1. Введение

Настоящий документ представляет собой **план тестирования с приложенными тест-кейсами** для сертифицируемого программного продукта «веб-сервис для подбора кандидатов» (см. техническое задание в разделе 2 настоящего отчёта). Документ предназначен для использования при проведении квалификационного тестирования и имитации сертификационных испытаний программного продукта.

План ориентирован на проверку выполнения ключевых функциональных и нефункциональных требований, изложенных в ТЗ, и предназначен для применения в учебной лабораторной среде.

2. Объект и цели тестирования

2.1 Объект тестирования

Объектом тестирования является веб-сервис для подбора кандидатов, реализованный как распределённое веб-приложение с микросервисной архитектурой и включающий, по крайней мере, следующие подсистемы:

- модуль управления пользователями (User/Auth Service);
- модуль загрузки и анализа резюме (ResumeService + MLAnalyzer);
- модуль управления вакансиями (VacancyService);
- модуль сопоставления кандидатов и вакансий (MatchingService);
- пользовательский веб-интерфейс (Frontend).

2.2 Цели тестирования

Основные цели тестирования:

- подтвердить, что реализованный функционал соответствует требованиям ТЗ к основным модулям системы;
- убедиться в корректности обработки типичных и граничных сценариев использования;
- проверить устойчивость системы к ошибочным и некорректным входным данным на уровне пользовательских и API-интерфейсов;
- оценить воспроизводимость результатов сопоставления (MatchingService) и корректность работы ключевых бизнес-сценариев;
- зафиксировать набор тест-кейсов, достаточный для проведения квалификационного тестирования в учебной среде.

3. Объем и границы тестирования

3.1 Входящие в объем тестирования компоненты

В тестирование включаются:

- функциональные требования к:
- регистрации, аутентификации и разграничению доступа пользователей;
- загрузке, хранению и обновлению резюме;
- созданию, редактированию и просмотру вакансий;
- вычислению меры соответствия и формированию ранжированного списка кандидатов;
- основным пользовательским сценариям работы кандидата и HR-специалиста;
- базовые нефункциональные аспекты:
- обработка ошибок и консистентность данных;
- приемлемое время отклика для типовых операций в учебной среде.

3.2 Исключения из объема

В рамках настоящего плана **не выполняется**:

- нагрузочное и стресс-тестирование под высокой массовой нагрузкой;
- формальная проверка безопасности уровня промышленной эксплуатации (пентест, анализ уязвимостей);
- глубинное тестирование DevOps-пайплайнов и инфраструктуры Kubernetes;
- детальная валидация качества ML-моделей (оценивается только базовая техническая интеграция).

4. Стратегия и виды тестирования

4.1 Функциональное тестирование (frontend + API)

Проводится по пользовательским сценариям и API-операциям:

- позитивные сценарии (валидные данные, ожидаемое поведение);
- негативные сценарии (некорректный формат, отсутствие обязательных полей, отсутствие прав доступа);
- проверки граничных значений (минимальные/максимальные длины полей, пустые коллекции и т.п.).

4.2 Модульное и интеграционное тестирование MatchingService

Для модуля сопоставления кандидатов (MatchingService) применяются:

- модульные/юнит-тесты ключевых методов (например, `CalculateSimilarity`, `RankCandidates`);

- интеграционные проверки работы MatchingService в составе цепочки «резюме → векторизация → сопоставление → рекомендации».

Подробное тестирование MatchingService опирается на наработки из лабораторной работы по тестированию модуля сопоставления (см. отчёт по ЛР3 «Тестирование MatchingService»).

4.3 Тестирование пользовательского интерфейса

Проверяются:

- корректность навигации между основными экранами;
- наличие и адекватность сообщений об ошибках;
- логичность и понятность форм и элементов управления для кандидата и HR-специалиста;
- согласованность отображаемых данных с результатами backend-операций.

5. Тестовое окружение

5.1 Программно-аппаратная среда

- операционная система сервера: Ubuntu Server LTS (или эквивалент в учебной среде);
- контейнерная платформа: Docker (при наличии — оркестрация Minikube/Kubernetes);
- СУБД: PostgreSQL;
- среда разработки и отладки: PyCharm/VS Code или эквивалент;
- браузер для проверки UI: современный браузер с поддержкой HTML5 (Chrome/Firefox/Edge).

5.2 Тестовые учетные записи и данные

Необходимо подготовить:

- минимум по одной тестовой учётной записи для ролей:
- кандидат,
- HR-специалист,
- администратор;
- набор тестовых резюме (от 3 до 5 штук) с различным набором навыков и опыта;
- набор тестовых вакансий (от 3 до 5 штук) с различными требованиями;
- заранее известные пары «резюме–вакансия» с ожидаемой высокой/низкой степенью соответствия для проверки MatchingService.

6. Критерии начала и окончания тестирования

6.1 Критерии начала (entry criteria)

- завершена реализация функционала, заявленного в ТЗ, в минимально работоспособном виде;
- система устанавливается и запускается в тестовой среде без критических ошибок;
- подготовлены тестовые данные и учётные записи;
- доступна документация ТЗ и архитектурная модель.

6.2 Критерии окончания (exit criteria)

- выполнены все тест-кейсы из настоящего плана;
- все дефекты, критичность которых не ниже «средней», либо исправлены, либо документированы с обоснованием;
- не осталось незакрытых блокирующих дефектов (срыв ключевых пользовательских сценариев);
- сформирован краткий отчёт о результатах тестирования и вывод о соответствии продукта ТЗ.

7. Структура набора тест-кейсов

Тест-кейсы сгруппированы по основным подсистемам:

1. Модуль управления пользователями (Auth/User).
2. Модуль загрузки и анализа резюме (Resume).
3. Модуль управления вакансиями (Vacancy).
4. Модуль сопоставления кандидатов и вакансий (MatchingService).
5. Пользовательский интерфейс (Frontend) — сквозные сценарии.

Каждый тест-кейс описывается в табличной форме:

- **ID** — уникальный идентификатор теста;
- **Название** — краткое описание цели теста;
- **Предусловия** — состояние системы и данные до начала теста;
- **Шаги** — последовательность действий;
- **Входные данные** — конкретные значения, вводимые пользователем или отправляемые в API;
- **Ожидаемый результат** — критерии успешного выполнения теста;
- **Тип** — функциональный/негативный/интеграционный;

- **Приоритет** — высокий (H), средний (M) или низкий (L).

8. Тест-кейсы по подсистемам (фрагменты)

В полном виде тест-кейсы приведены в исходном плане тестирования. Ниже приведены репрезентативные фрагменты таблиц для отдельных подсистем (AUTH, RES, VAC, MAT, UI), демонстрирующие структуру и охват.

(далее идут таблицы тест-кейсов из исходного плана; в отчёте они сохраняются без изменений)

9. Оценка полноты и достаточности тестирования

Предложенный набор тест-кейсов охватывает:

- ключевые функциональные требования ТЗ к основным модулям системы;
- типичные пользовательские сценарии работы кандидата и HR-специалиста;
- базовые негативные и граничные ситуации (отказ в доступе, некорректные данные, временная недоступность сервисов);
- воспроизводимость результатов сопоставления в MatchingService.

В учебной конфигурации данного набора тестов достаточно для проведения квалификационного тестирования и формирования мотивированного заключения о соответствии веб-сервиса заявленным требованиям ТЗ.

10. Заключение по плану тестирования

Настоящий план тестирования с приложенными тест-кейсами может использоваться:

- как основа для ручного прогона тестов в лабораторной среде;
- как база для реализации автоматизированных модульных и интеграционных тестов (на уровне кода и API);
- как комплект материалов, предъявляемых при имитации процедуры сертификации программного продукта.

Результаты выполненных по данному плану испытаний, а также выявленные дефекты и их анализ должны быть отражены в отчёте по лабораторной работе и в заключении о соответствии программного продукта требованиям технического задания.

5. Оценка полноты плана тестирования по составу

Оценка полноты плана тестирования по составу тестов

1. Объект и цель оценки

Объектом оценки является план тестирования веб-сервиса подбора кандидатов, разработанный в рамках лабораторной работы №3 по тестированию модуля сопоставления кандидатов и вакансий (`MatchingService` , класс `MatchEngine` , функции `CalculateSimilarity` и `RankCandidates`) и распространяемый на остальные ключевые модули, определённые в техническом задании на сертифицируемый продукт.

Цель оценки — установить, насколько состав тестов (по типам, видам и охвату функциональности) является достаточным для квалификационного тестирования продукта и формирования мотивированного заключения о его соответствии требованиям ТЗ на веб-сервис подбора кандидатов.

При оценке полноты по составу тестов учитываются как разработанные ранее тесты для `MatchingService` (см. ЛР3 «Тестирование `MatchingService`»), так и запланированное расширение тестового покрытия на остальные модули, указанные в ТЗ на сертифицируемый продукт.

2. Классификация тестов, предусмотренных планом

В плане тестирования предусмотрены следующие основные виды тестов:

- **Элементарные (примерные) unit-тесты**

Предназначены для точечной проверки корректности ключевых функций на типичных и граничных входах. Для `MatchingService` реализованы тесты, проверяющие:

- идентичные, ортогональные и противоположные векторы признаков;
- обработку нулевых векторов и пустых входов;
- реакцию на различную длину векторов (ошибочный вход);
- корректность ранжирования кандидатов, наличие явного лидера, устойчивость порядка при равенстве оценок, обработку пустого списка кандидатов.

- **Тесты, управляемые данными (data-driven unit-тесты)**

Используют внешние источники данных (`data_matching.xml` , `data_matching.csv`) и единый тестовый код, последовательно прогоняемый по строкам набора данных. Такие тесты позволяют:

- масштабировать количество проверок без дублирования кода;

- фиксировать ожидаемые значения сходства и порядок ранжирования для множества случаев;
- проводить регрессионное тестирование при изменении реализации.
- **Функциональные тесты уровня API**
Ориентированы на проверку сценариев использования, описанных в ТЗ (регистрация и аутентификация, загрузка резюме, управление вакансиями, получение рекомендаций). Для веб-сервиса подбора кандидатов такие тесты предполагают:
 - проверку корректной обработки запросов к REST-интерфейсам (успешные и ошибочные случаи);
 - валидацию входных данных и сообщений об ошибках;
 - проверку целостности цепочки «резюме → анализ → сопоставление → рекомендация».
- **Негативные тесты и тесты устойчивости к ошибкам**
Направлены на проверку реакции системы на некорректные входы и сбои внешних компонент:
 - неподдерживаемые форматы файлов резюме, повреждённые документы;
 - некорректные или неполные данные в вакансиях;
 - отсутствие кандидатских профилей для сопоставления;
 - сетевые ошибки при межсервисном взаимодействии.

Состав тестов охватывает как внутреннюю логику ключевого модуля сопоставления, так и интеграционные аспекты, вытекающие из архитектуры микросервисного веб-сервиса.

3. Покрытие функциональных требований ТЗ

В соответствии с ТЗ на сертифицируемый продукт веб-сервис подбора кандидатов включает следующие основные функциональные модули: управление пользователями (User/Auth Service), загрузка и анализ резюме (ResumeService + MLAnalyzer), управление вакансиями (VacancyService), сопоставление кандидатов и вакансий (MatchingService) и пользовательский интерфейс (Frontend).

Оценка покрытия по составу тестов:

- **Модуль сопоставления кандидатов и вакансий (MatchingService)**
- Элементарные unit-тесты и data-driven тесты обеспечивают проверку:
 - корректности вычисления меры сходства в диапазоне [0; 1] для репрезентативных классов эквивалентности и граничных значений;
 - устойчивости алгоритма ранжирования, детерминированности результата, стабильности порядка при равенстве оценок;

- обработки некорректных входов (разная длина векторов, пустые списки кандидатов).
- Таким образом, функциональные требования ТЗ к MatchingService (см. раздел 5.4 ТЗ) покрываются по основным сценариям и граничным случаям.
- **Модуль загрузки и анализа резюме (ResumeService + MLAnalyzer)**
- Планом тестирования предусмотрены функциональные тесты API на:
 - приём резюме в поддерживаемых форматах (PDF, DOCX);
 - корректную реакцию на неподдерживаемые или повреждённые файлы;
 - валидацию обязательных полей структурированного профиля (образование, опыт, навыки).
- Планируется дополнение набора тестов проверками корректности векторизации (на уровне формата и диапазона значений), однако детальная проверка качества ML-модели выходит за рамки квалификационного тестирования и фиксируется как ограничение.
- **Модуль управления вакансиями (VacancyService)**
- Включён набор тестов CRUD-операций (создание, изменение, просмотр, архивирование), проверяющих:
 - обязательность полей вакансии, корректность типов значений;
 - корректную смену статусов вакансий (активна, приостановлена, закрыта);
 - доступность данных вакансий для сопоставления и отображения в интерфейсе.
- Набор тестов признаётся достаточным для проверки заявленных в ТЗ функциональных требований к управлению вакансиями.
- **Модуль управления пользователями (User/Auth Service)**
- Планом предусмотрены тесты:
 - регистрации и аутентификации пользователей разных ролей;
 - отказа в доступе при неверных учётных данных;
 - разграничения прав доступа к операциям по ролям (кандидат, HR-специалист, администратор).
- На уровне текущей лабораторной работы состав таких тестов оценивается как достаточный для подтверждения базовой безопасности и корректности управления доступом, заявленных в ТЗ.
- **Пользовательский интерфейс (Frontend)**
- Предусмотрены сценарные функциональные тесты ключевых пользовательских потоков:
 - загрузка и обновление резюме кандидатом;

- создание вакансии HR-специалистом и просмотр списка кандидатов;
- просмотр рекомендаций и переход к карточке кандидата.
- Полнота UI-тестирования оценивается как базовая; для учебных целей этого достаточно, однако для промышленной сертификации требовалось бы расширение набора сценариев и кросс-браузерное тестирование.

В целом, план тестирования по составу тестов обеспечивает покрытие основных функциональных требований ТЗ и ключевых пользовательских сценариев.

4. Покрытие нефункциональных требований

Нефункциональные требования ТЗ включают надёжность, производительность, безопасность, требования к среде функционирования и удобству использования.

Оценка предусмотренного состава тестов:

- **Надёжность и отказоустойчивость**
- Набор тестов включает проверки корректной обработки ошибок ввода, некорректных форматов файлов, отсутствия данных и отдельных сбоев внешних сервисов (на уровне имитации отказов).
- При этом полноформатное нагрузочное и стресс-тестирование не входит в текущий состав тестов и фиксируется как ограничение плана.
- **Производительность**
- В план включены ориентировочные проверки времени отклика типичных операций (получение рекомендаций, загрузка резюме) в учебной среде.
- Детальные нагрузочные тесты с моделированием реальной эксплуатационной нагрузки не разрабатывались, что обусловлено учебным характером работы.
- **Безопасность**
- Тесты управления доступом и аутентификации проверяют базовые аспекты безопасности: запрет неавторизованного доступа, корректность разграничения по ролям, безопасную обработку некорректных запросов.
- Специализированные тесты на уязвимости (SQL-инъекции, XSS, CSRF) в план не включены и отмечаются как резерв для расширения.
- **Удобство использования (эргономика)**
- В рамках учебного проекта предусмотрены ручные проверочные сценарии на понятность интерфейса и читаемость сообщений об ошибках, однако формализованные юзабилити-тесты не разрабатывались.

Таким образом, состав тестов даёт минимально необходимое подтверждение выполнения ключевых нефункциональных требований ТЗ, но для промышленной

сертификации потребовал бы значительного расширения, прежде всего в части нагрузочного тестирования и безопасности.

5. Анализ достаточности и выявленные пробелы

На основе сопоставления плана тестирования с требованиями ТЗ и методическими указаниями к лабораторной работе можно сформулировать следующие выводы о полноте по составу тестов:

- Для ядра интеллектуальной логики (MatchingService) состав unit- и data-driven тестов является **достаточным** для учебной квалификационной оценки:
- покрыты основные классы эквивалентности и граничные случаи;
- реализована проверка устойчивости сортировки и обработки некорректных входов;
- обеспечена возможность расширения набора кейсов за счёт внешних файлов данных.
- Для функциональных модулей, описанных в ТЗ (управление пользователями, резюме, вакансиями, пользовательский интерфейс), разработан **базовый** набор тестов, покрывающий:
- типичные позитивные сценарии использования;
- часть негативных сценариев (ошибки ввода, отсутствие данных, отказ в доступе).
- В плане тестирования зафиксированы следующие **пробелы**, снижающие полноту по составу тестов:
- отсутствие систематического нагрузочного тестирования и тестов масштабируемости;
- отсутствие специализированных тестов безопасности (пентест-кейсы, проверки типовых веб-уязвимостей);
- ограниченное количество формализованных сценариев для пользовательского интерфейса и эргономики;
- отсутствие комплексных сквозных тестов, проверяющих полный путь данных от загрузки резюме до решения по кандидату с фиксацией результата.

Указанные пробелы учитываются как ограничения учебного формата работы и не препятствуют выполнению целей лабораторной работы, но должны быть явно отражены в заключении о сертификации.

6. Итоговый вывод

С учётом анализа требований технического задания на веб-сервис подбора кандидатов и сопоставления их с фактическим составом разработанных тестов делается следующий вывод:

- по **ключевому модулю сопоставления кандидатов и вакансий (MatchingService)** план тестирования по составу тестов можно считать **полным и достаточным** для целей учебного квалификационного тестирования;
- по **остальным функциональным и нефункциональным требованиям ТЗ** состав тестов является **базовым** и обеспечивает минимально необходимое подтверждение корректности работы системы в основных сценариях, но не покрывает в полном объёме аспекты производительности, безопасности и удобства использования.

В рамках лабораторной работы №4 такой уровень полноты плана тестирования по составу тестов оценивается как **достаточный для формирования мотивированного заключения** о соответствии реализованного прототипа веб-сервиса подбора кандидатов заявленным в ТЗ требованиям, при обязательной фиксации выявленных ограничений и допущений.

6. Оценка полноты плана тестирования по продолжительности

Оценка полноты плана тестирования по продолжительности

1. Исходные данные

План тестирования разрабатывается для сертифицируемого веб-сервиса подбора кандидатов, описанного в техническом задании на продукт (см. ТЗ на сертифицируемый продукт, веб-сервис подбора кандидатов). Объект тестирования включает функциональные модули (управление пользователями, загрузка и анализ резюме, управление вакансиями, сопоставление кандидатов и вакансий, пользовательский интерфейс) и нефункциональные характеристики (надёжность, производительность, безопасность, воспроизводимость развертывания).

Продолжительность тестирования планируется в рамках учебной лабораторной работы и ограничена доступным временем семестра, но при этом должна обеспечивать:

- как минимум один полный цикл функционального тестирования по всем приоритетным сценариям;
- не менее двух регрессионных прогонов по критически важным функциям после доработок;
- хотя бы один целевой прогон нефункциональных проверок (производительность на учебных нагрузках, устойчивость к отказам среды, воспроизводимость развертывания).
- Структура и длительность этапов тестирования

В плане тестирования фиксируются следующие этапы и ориентировочные трудозатраты.

2.1. Подготовка среды и исходных данных

- Развёртывание тестовой конфигурации веб-сервиса в контейнерной среде (Docker/Minikube), настройка БД PostgreSQL.
- Подготовка учебного набора данных: резюме кандидатов, вакансии различных типов, учётные записи пользователей целевых ролей.
- Проверка соответствия среды требованиям эксплуатации, указанным в ТЗ, на уровне учебного стенда.

Ориентировочная длительность: 4–6 часов. Этого достаточно для базовой настройки кластера/контейнеров и наполнения тестовой базы типовыми данными с учётом ограниченного масштаба учебной системы.

2.2. Разработка и верификация тестов

- Детализация сценариев из плана тестирования в виде тест-кейсов для ключевых функций:
- регистрация и аутентификация пользователей;
- загрузка и обновление резюме;
- создание и сопровождение вакансий;
- получение рекомендаций кандидатов по вакансии (MatchingService);
- сценарии отказов (некорректные форматы файлов, недоступность сервисов, ошибки авторизации).
- Подготовка наборов входных данных и ожидаемых результатов.
- Верификация тест-кейсов на непротиворечивость и воспроизводимость.

Ориентировочная длительность: 6–8 часов. При таком объёме времени возможно сформировать 20–30 тест-кейсов, покрывающих все критические сценарии из ТЗ и плана тестирования, включая позитивные и граничные случаи.

2.3. Первый полный прогон функциональных тестов

- Последовательное выполнение всех тест-кейсов функционального уровня:
- базовые сценарии работы кандидата и HR-специалиста;
- сценарии с отказами и некорректными входами;
- проверки целостности данных в БД после типовых операций.
- Фиксация результатов в протоколах тестирования и дефектных отчётах.

Ориентировочная длительность: 6–8 часов. Этого времени достаточно для полного прогона подготовленного набора тестов в ручном режиме в условиях учебной нагрузки, включая время на повторные проверки спорных случаев.

2.4. Нефункциональные проверки

- Проверка времени отклика типовых операций при учебной нагрузке;
- загрузка резюме;
- получение рекомендаций для вакансии с несколькими десятками кандидатов;
- просмотр списков вакансий и резюме.
- Наблюдение за устойчивостью работы системы при кратковременных отказах отдельных сервисов (например, недоступность MatchingService или БД) в пределах возможностей учебного стенда.
- Проверка воспроизводимости развертывания тестовой среды по подготовленным инструкциям.

Ориентировочная длительность: 4–5 часов. В рамках этого времени возможно провести несколько серий измерений на учебных объёмах данных и зафиксировать результаты в отчёте, не претендуя на промышленный нагрузочный профиль.

2.5. Регрессионное тестирование после исправления дефектов

- Повторный прогон всех тест-кейсов, затронутых исправлениями.
- Полный выборочный прогон по критическим бизнес-процессам:
- регистрация и вход;
- загрузка и обновление резюме;
- создание вакансии и получение рекомендаций;
- фиксация решения по кандидату.
- Контроль отсутствия деградации ранее работавших функций.

Ориентировочная длительность: 4–6 часов на два регрессионных цикла (по 2–3 часа каждый). Этого достаточно для подтверждения устойчивости критических сценариев после доработок.

2.6. Анализ результатов и оформление выводов

- Сводная оценка выполнения тест-кейсов и закрытия дефектов.
- Оценка соответствия продукта требованиям ТЗ и критериям сертификации.
- Подготовка итогового отчёта по лабораторной работе с приложением протоколов тестирования.

Ориентировочная длительность: 3–4 часа. В это время укладывается подготовка структурированного заключения по результатам всех прогонов.

1. Оценка достаточности продолжительности тестирования

3.1. Функциональное покрытие во времени

Суммарное время, отводимое на разработку тестов и их исполнение (пункты 2.2, 2.3 и 2.5), составляет порядка 16–22 часов. При таком объёме трудозатрат:

- все функции, указанные в ТЗ и отражённые в плане тестирования, могут быть проверены не менее одного раза в рамках первого полного прогона;

- для критических сценариев подбора кандидатов, влияющих на качество рекомендаций, и для операций изменения данных (создание/обновление резюме и вакансий) закладывается не менее двух регрессионных прогонов;
- остаётся резерв времени на уточнение тест-кейсов по результатам первых прогонов и на проверку исправлений.

Таким образом, с точки зрения функциональной полноты тестирования планируемая продолжительность обеспечивает выполнение требований лабораторной работы по количеству и глубине проверок.

3.2. Нефункциональные характеристики

Временной ресурс, выделенный на нефункциональные проверки (4–5 часов), достаточен для:

- оценки приемлемости времени отклика системы на учебных объёмах данных;
- проверки поведения системы при типовых отказах отдельных компонентов в пределах одной тестовой сессии;
- подтверждения воспроизводимости развертывания по подготовленным инструкциям.

При этом полнота нагрузочного тестирования в промышленном смысле ограничена: в план не включаются длительные многочасовые прогоны под высокой нагрузкой и моделирование пиковых сценариев эксплуатации. Такие испытания признаются выходящими за рамки учебной лабораторной работы и фиксируются как ограничение.

3.3. Временные риски и допущения

В рамках принятого плана предполагается, что:

- количество обнаруженных дефектов будет умеренным и не потребует более двух итераций исправлений и регресса;
- существенных архитектурных изменений после начала тестирования не планируется;
- подготовка среды и данных не потребует дополнительных непредвиденных трудозатрат.

При нарушении этих допущений (например, при необходимости серьёзной переработки архитектуры или миграции среды) заложенное время может оказаться недостаточным, что отразится на полноте тестирования по длительности. Данный риск учитывается в экспертном заключении по результатам лабораторной работы.

1. Вывод

С учётом структуры этапов, распределения трудозатрат и ограничений учебного формата продолжительность тестирования, заложенная в плане, можно считать достаточной для:

- выполнения полного функционального прогона по всем критическим сценариям веб-сервиса подбора кандидатов;
- проведения как минимум двух регрессионных циклов по ключевым бизнес-процессам;
- выполнения базового анализа производительности и устойчивости на учебных нагрузках;
- подготовки обоснованного экспертного заключения о соответствии продукта требованиям ТЗ.

Полнота плана тестирования по продолжительности является достаточной для целей квалификационного тестирования и имитации сертификационных испытаний в рамках лабораторной работы, при признании отмеченных ограничений по глубине нагрузочного тестирования.

7. Оценка качества документирования тестов и их воспроизводимости

Оценка качества документирования тестов и их воспроизводимости

1. Объект оценки и используемые артефакты

Объектом оценки является набор материалов по тестированию модуля MatchingService веб-сервиса подбора кандидатов, разработанный в рамках лабораторной работы №3 «Тестирование модуля MatchingService». В работу входят:

- спецификация тестируемого модуля (назначение, функции, интерфейсы);
- план тестирования с описанием подходов (unit-тесты и data-driven);
- набор тест-кейсов с входами и ожидаемыми результатами;
- примеры реализаций unit-тестов на MSTest;
- описание и структура файлов данных (data_matching.xml , data_matching.csv);
- описание запуска тестов и протоколов выполнения.

Оценка качества документирования проводится с точки зрения того, насколько по этим материалам внешний эксперт, не участвовавший в разработке, может:

- понять цели и границы тестирования;
- воспроизвести тесты на своей машине;
- интерпретировать результаты прогона и связать их с требованиями к модулю.

2. Структура и полнота документирования тестов

2.1. Описание тестируемого модуля

В разделе «Спецификация модуля MatchingService» (см. документ по ЛР3 «Тестирование модуля MatchingService») дано:

- формальное назначение модуля (сопоставление кандидатов и вакансий по векторным представлениям);
- перечень функций (расчёт косинусного сходства, ранжирование кандидатов, обработка граничных случаев);
- место модуля в архитектуре (взаимодействие с ResumeService , VacancyService , MLAnalyzer);
- чётко выделенные интерфейсы для тестирования:
- `double CalculateSimilarity(double[] v1, double[] v2) ,`
- `List<(candidateId: string, score: double)> RankCandidates(double[] vacancy, Dictionary<string, double[]> candidates) .`

Такое описание считается достаточным для понимания контекста тестов: указываются и функциональные обязанности, и границы ответственности, и ожидаемые свойства (диапазон значений, устойчивость к ошибочным входам). Это повышает трассируемость между требованиями и тест-кейсам.

2.2. План тестирования и выбранные подходы

В документе по ЛР3 явно зафиксирован выбор двух подходов:

- элементарные unit-тесты (примерные) для точечных сценариев;
- тесты, управляемые данными (data-driven) с источниками XML/CSV.

План тестирования включает:

- цели тестирования (проверка корректности метрики, ранжирования, устойчивости к граничным случаям);
- техники проектирования тестов (классы эквивалентности, граничные значения, позитивные и негативные случаи);
- область покрытия для двух функций (CalculateSimilarity , RankCandidates);
- обоснование количества тестов (раздел «План тестирования (обоснование количества и состава тестов)»).

Структура плана согласована с методическими требованиями к ЛР3 и соответствует п. «план тестирования с приложенными тестами» из ЛР4: цели, методы, покрытие и критерии приёмки формализованы. В результате качество документирования на уровне плана можно оценить как **высокое**.

2.3. Документирование тест-кейсов

Тест-кейсы описаны в текстовой форме и в табличном виде.

- Для `CalculateSimilarity` приведены шесть базовых сценариев (идентичные, ортогональные, противоположные векторы, нулевой вектор, разная длина, дробные нормированные значения) с указанием:
 - входных данных (конкретные векторы);
 - ожидаемого результата (число или исключение);
 - краткого обоснования.
- Для `RankCandidates` выделены сценарии:
 - явный лидер по сходству;
 - равные `score` и проверка стабильности сортировки;
 - пустой список кандидатов;
 - кандидат с нулевым вектором;
 - смешанные значения с близкими `score`.

Также представлен табличный фрагмент набора данных (`Caseld`, `Vacancy`, `CandidateId`, `Resume`, `ExpectedSimilarity`, `ExpectedOrder`) и эскиз XML-структуры. Это обеспечивает машиночитаемое и человекочитаемое описание тестов.

К недостаткам можно отнести:

- ограниченность примеров в табличном фрагменте (показаны только первые строки, без полного перечня набора);
- отсутствие в отдельном приложении полного файла `data_matching.xml` в виде приложенного к отчёту приложения (сейчас он описан текстом и упомянут по пути).

В целом, описание тест-кейсов систематизировано, формат повторяем, связи с требованиями явно указаны. Качество документирования тест-кейсов оценивается как **выше среднего** с потенциалом улучшения за счёт вынесения полного набора данных в приложение к отчёту ЛР4.

3. Документирование реализации и окружения тестов

3.1. Реализация тестов (MSTest)

В отчёте приведены примеры:

- интерфейса `IMatchEngine` и учебной реализации `MatchEngine` ;
- базового набора тестов `UnitTest_MatchingServiceBasics` с атрибутами `[TestMethod]` , `[TestCategory]` ;
- реализации data-driven тестов `UnitTest_MatchingService_DataDriven` с использованием `TestContext` , `[DeploymentItem]` и `[DataSource]` .

Код снабжён минимальными, но достаточными комментариями по назначению каждого тестового метода и ожидаемому поведению. Стиль кодирования выдержан, имена методов отражают проверяемый сценарий

(`Similarity_IdenticalVectors_IsOne` , `RankCandidates_LeaderFirst` и т.п.), что упрощает анализ протоколов выполнения.

Уровень документирования реализации тестов можно оценить как **хороший**: приведены репрезентативные фрагменты, показаны шаблоны подключения источников данных и вспомогательных функций (`ParseVector`).

3.2. Описание окружения и инструментов

Раздел «Подготовка окружения: VS 2022 + MSTest» содержит:

- требования к среде разработки (Visual Studio 2022, .NET SDK, MSTest);
- варианты целевых фреймворков (.NET 6/7/8 либо .NET Framework 4.8);
- структуру проекта (папки `Core` , `Data` , `UnderTest` , `Tests`);
- пошаговую инструкцию создания решения через GUI VS и через `dotnet CLI`;
- указания по настройке свойств файлов данных (Build Action, Copy to Output Directory);
- пример команды запуска тестов `dotnet test ...` и выдержку из логов.

Такое описание позволяет внешнему эксперту без существенных догадок поднять аналогичное окружение и запустить тесты. Отдельно зафиксированы критерии входа (проект собирается, данные доступны) и выхода (все тесты зелёные, допуск по погрешности).

К возможным улучшениям можно отнести:

- явное указание версии .NET и MSTest, на которой тесты были фактически выполнены;
- краткий список внешних зависимостей (если бы они появлялись) с ссылкой на используемые пакеты.

Тем не менее, для учебных целей текущий уровень детализации достаточен. Качество документирования окружения оценивается как **высокое**.

4. Оценка воспроизводимости тестов

4.1. Формальные критерии воспроизводимости

Под воспроизводимостью в контексте лабораторной работы понимается возможность:

- собрать исходный проект и его тестовый проект на другой машине;
- выполнить все указанные тесты без ручной модификации кода;
- получить результаты, совпадающие с ожидаемыми в пределах заявленной числовой погрешности;
- однозначно интерпретировать результаты по именам тестов и логам.

4.2. Факторы, повышающие воспроизводимость

В представленных материалах присутствуют следующие позитивные факторы:

- **Детерминированная логика:** тестируемый код (MatchEngine) не использует случайность, внешние сервисы или время; все входы заданы явно.
- **Чётко зафиксированные входы и ожидания:** векторы и ожидаемые значения (числа и порядок) задокументированы и в описании тест-кейсов, и в исходном коде тестов.
- **Ограничение погрешности:** в assertions для similarity явно указан допуск (1e-9 для элементарных тестов, 1e-4 для data-driven), что позволяет избежать ложных падений из-за особенностей плавающей арифметики.
- **Указание команд запуска и пример логов:** приведена команда dotnet test и выдержка из логов успешного прогона, что задаёт эталон для сравнения.
- **Структура данных для data-driven:** формат XML/CSV и имена полей стабильны и описаны в документации.

С учётом этих факторов, при наличии исходного кода и файлов данных другой студент или преподаватель сможет повторить запуск и получить те же результаты.

4.3. Потенциальные риски и ограничения

Несмотря на общий высокий уровень, есть ряд предпосылок, снижающих формальную воспроизводимость:

- не зафиксирована конкретная версия .NET и Visual Studio (указан диапазон допустимых версий, но не «фактически использованная» конфигурация);
- нет явного описания структуры каталогов решения (например, относительных путей от корня репозитория), хотя это частично покрыто инструкциями по созданию;
- файл data_matching.xml описан как существующий, но его полное содержимое не включено в отчёт отдельным приложением (рассчитано на доступ к репозиторию).

Эти факторы не блокируют воспроизведение в учебной среде, но могут потребовать дополнительных уточнений при независимой сертификации.

В совокупности воспроизводимость тестов оценивается как **хорошая**: при соблюдении общих версий инструментов и наличии файлов данных все проверки могут быть надёжно повторены.

5. Сводная оценка и рекомендации по улучшению

5.1. Сводная качественная оценка

С учётом требований ЛР4 к «оценке качества документирования тестов и их воспроизводимости» и анализа материалов по ЛР3, можно дать следующую обобщённую оценку:

- **Полнота документирования:** тестируемый модуль, его интерфейсы, цели тестирования и область покрытия описаны явно; план тестирования структурирован. Уровень — **высокий**.
- **Ясность и структурированность:** формат описания тест-кейсов однороден, используется комбинация текстового и табличного представления, имена методов тестов само-документирующие. Уровень — **высокий**.
- **Трассируемость к требованиям:** связь между требованиями (корректность косинусного сходства, устойчивость ранжирования) и отдельными тестами прослеживается, но могла бы быть усилена явной матрицей соответствия «требование → тест-кейс». Текущий уровень — **выше среднего**.
- **Воспроизводимость:** при наличии исходников и файлов данных тесты могут быть повторно запущены по описанной инструкции; риски связаны в основном с нефиксированными версиями инструментов. Уровень — **хороший**.

В терминах квалификационной проверки, пакет материалов по тестированию модуля MatchingService можно считать **достаточно документированным и воспроизводимым** для учебной сертификации.

5.2. Рекомендации по доработке

Для повышения качества документирования и воспроизводимости тестов при использовании материалов в рамках ЛР4 целесообразно:

1. Включить в отчёт отдельное приложение с полным содержимым файлов data_matching.xml и data_matching.csv либо дать на них явную ссылку в структуре репозитория.
2. Добавить краткую матрицу трассируемости вида «функциональное требование / свойство → набор тест-кейсов», чтобы упростить экспертную оценку полноты покрытия.
3. Зафиксировать конкретную комбинацию версий инструментов (например, «.NET 8.0, MSTest vX.Y, Visual Studio 2022 17.Z») и указать её в разделе «Окружение».
4. При необходимости реальной сертификации дополнить документацию ссылкой на систему контроля версий (идентификатор коммита решения, в котором тесты были выполнены).

Реализация указанных рекомендаций позволит перейти от учебного уровня документирования к уровню, более близкому к промышленным практикам квалификационного тестирования и сертификационных испытаний.

8. Оценка адекватности среды целям проведённого тестирования

1. Введение

Настоящий раздел посвящён оценке адекватности среды тестирования целям проведённого квалификационного тестирования веб-сервиса для подбора кандидатов. Объект тестирования и сертификации определяется техническим заданием на сертифицируемый программный продукт (см. соответствующий раздел отчёта по лабораторной работе № 4), а также проектной моделью, разработанной в предыдущем семестре. Целью оценки является проверка того, что выбранная и фактически использованная среда тестирования позволяет достоверно судить о соответствии продукта требованиям технического задания и условиям предполагаемой эксплуатации.

2. Цели и задачи тестирования

Основные цели квалификационного тестирования веб-сервиса:

- проверка выполнения функциональных требований технического задания (регистрация и аутентификация пользователей, загрузка и хранение резюме, управление вакансиями, вычисление меры соответствия и ранжирование кандидатов, работа пользовательского интерфейса);
- проверка ключевых нефункциональных требований в учебном объёме (надёжность обработки ошибок, воспроизводимость результатов, приемлемое время отклика в условиях лабораторной нагрузки);
- получение оснований для формирования экспертного заключения о готовности продукта к сертификационным испытаниям.

Из этих целей следуют требования к среде тестирования:

- достаточная близость к целевой программно-аппаратной платформе, указанной в ТЗ (Linux-сервер, контейнеризация, PostgreSQL, веб-сервер/обратный прокси);
- наличие инструментов для воспроизводимого развёртывания и конфигурирования системы;
- возможность имитации типичных сценариев эксплуатации с точки зрения всех акторов (кандидат, HR-специалист, администратор).

3. Характеристика целевой среды эксплуатации

В техническом задании целевая среда функционирования веб-сервиса описана следующим образом:

- серверная платформа: дистрибутив Linux серверного класса (Ubuntu Server LTS);
- система управления базами данных: PostgreSQL;

- архитектура приложения: набор микросервисов (backend-сервисы, ML-компонент, frontend) с использованием Docker и, при развёртывании промышленного уровня, оркестратора Kubernetes;
- веб-сервер/обратный прокси: Nginx или эквивалент;
- клиентская часть: современный веб-браузер, работающий по протоколу HTTPS.

Целевая среда предполагает эксплуатацию в условиях многопользовательского доступа, с возможностью масштабирования отдельных микросервисов, а также интеграцию с инфраструктурой DevOps (средства контроля версий, CI/CD). При этом в рамках учебного проекта допускается использование упрощённой конфигурации, при условии сохранения ключевых характеристик архитектуры и стека технологий.

4. Описание фактической среды тестирования

В рамках лабораторной работы №4 среда тестирования была построена как упрощённая, но архитектурно согласованная с целевой:

- **Операционная система и хост-среда.** Для развёртывания продукта использовалась рабочая станция студента под управлением настольной ОС (семейство Windows или Linux), на которой были установлены Docker и инструменты разработки. Роль серверной ОС уровня Ubuntu Server выполняла среда контейнеров, базирующаяся на образах Linux.
- **Контейнеризация и оркестрация.** Основные компоненты веб-сервиса (backend, база данных PostgreSQL, при наличии — ML-модуль, web-сервер) запускались в изолированных Docker-контейнерах. Для управления многоконтейнерной конфигурацией применялся Docker Compose, обеспечивающий однокомандное развёртывание тестовой инсталляции. Полноценный кластер Kubernetes в рамках лабораторных испытаний не поднимался; его роль частично имитировалась средствами Compose (описание зависимостей, перезапуск контейнеров, проброс портов).
- **СУБД.** В качестве системы управления базами данных использовалась PostgreSQL, развёрнутая в контейнере. Структура схемы БД (пользователи, резюме, вакансии, отклики, служебные таблицы) соответствовала модели, заложенной в техническом задании и архитектурной документации.
- **Сервер приложений и frontend.** Backend-часть была развёрнута как веб-приложение (например, Django-сервис с REST API) с публикацией HTTP-эндпойнтов либо напрямую через встроенный сервер разработки, либо через связку с Nginx в контейнере. Клиентская часть (React/Vue-приложение) запускалась как статический frontend, обслуживаемый либо отдельным контейнером, либо локальным dev-сервером. Доступ к системе осуществлялся из браузера студента.
- **Инструменты тестирования.** Для модульного тестирования и частично для автоматизации регрессионных проверок применялись средства unit-тестирования

backend-логики и тестов для MatchingService, разработанных в предыдущей лабораторной работе. Для ручного функционального тестирования использовались:

- веб-браузер (проверка пользовательских сценариев);
- средства тестирования REST API (Postman или аналог) для точечных проверок интерфейсов микросервисов;
- средства просмотра логов контейнеров (docker logs).

Таким образом, фактическая среда тестирования воспроизводила распределённую структуру приложения (микросервисы, отдельная СУБД, фронтенд) и обеспечивала достаточный контроль над конфигурацией компонентов.

5. Соответствие среды критериям квалификационного тестирования

5.1 Функциональная полнота

Квалификационное тестирование требовало проверки всех основных сценариев использования, изложенных в техническом задании: регистрация и аутентификация, загрузка и обновление резюме, создание и сопровождение вакансий, запуск сопоставления кандидатов и вакансий, просмотр результатов в интерфейсе. Наличие развёрнутого backend, функционирующей БД PostgreSQL и доступного frontend-интерфейса обеспечило возможность полноценно проиграть все ключевые сценарии с точки зрения трёх ролей пользователей.

Интерфейсы микросервисов были доступны как через пользовательский интерфейс, так и через REST-клиенты, что позволило при необходимости подтвердить корректность обработки граничных и ошибочных случаев, не всегда удобно воспроизводимых из браузера. Таким образом, с точки зрения охвата функциональных требований среда тестирования была адекватной целям работы.

5.2 Воспроизводимость и управляемость

Использование Docker и Docker Compose позволило зафиксировать конфигурацию сервисов в виде файлов описания (YAML-манифесты для контейнеров), что обеспечивает воспроизводимость тестовой среды на других машинах с минимальными изменениями. Пере-развёртывание окружения (остановка, пересоздание контейнеров, инициализация БД) выполнялось стандартными командами и не зависело от специфики локальной установки библиотек.

Такая организация среды соответствует требованиям к воспроизводимости квалификационных испытаний и в учебном масштабе имитирует принципы промышленного развертывания (инфраструктура как код). Управляемость

конфигурации и возможность её фиксации в системе контроля версий дополнительно повышают доверие к результатам тестирования.

5.3 Близость к целевой архитектуре

Несмотря на упрощение (отсутствие полнофункционального кластера Kubernetes и высоконагруженного окружения), тестовая среда сохраняла ключевые черты целевой архитектуры:

- использование Linux-контейнеров в качестве среды выполнения микросервисов;
- наличие отдельной СУБД PostgreSQL;
- чёткое разделение frontend и backend;
- доступ к системе по HTTP(S)-протоколу из браузера.

С точки зрения логики взаимодействия компонентов (REST-API, формат JSON, обработка запросов и ответов) тестовая среда эквивалентна целевой. Потенциальные различия связаны в основном с масштабом (число пользователей, объём данных) и средствами оркестрации (Compose вместо Kubernetes), что критично для нагрузочного и отказоустойчивого тестирования, но менее существенно для проверки функциональной корректности и базовой надёжности.

5.4 Ограничения по нагрузке и масштабируемости

В лабораторных условиях нагрузка на систему ограничивалась малым числом одновременных пользователей (по сути, одним тестирующим) и небольшим тестовым набором данных (десятки резюме и вакансий). Это не позволяет сделать строгие выводы о поведении системы при промышленной нагрузке, а также о корректности автоматического масштабирования и балансировки нагрузки на уровне Kubernetes-кластера.

Тем не менее, с учётом целей лабораторной работы, основной акцент делался на корректности функционала и воспроизводимости результатов. Для этих целей выбранная среда является достаточной. Расширение испытаний до полноценного нагрузочного тестирования потребовало бы развертывания кластера Kubernetes и генератора нагрузки, что выходит за рамки данного этапа.

5.5 Соответствие требованиям безопасности

В учебной тестовой среде использовались упрощённые настройки безопасности: в ряде случаев доступ осуществлялся по HTTP в локальной сети, токены аутентификации и секреты могли храниться в конфигурационных файлах без применения специализированных хранилищ секретов. При переходе к промышленной эксплуатации в соответствии с ТЗ обязательны:

- использование HTTPS (TLS-сертификаты, настройка Nginx/Ingress-контроллера);

- централизованное хранение секретов (Kubernetes Secrets, внешние секрет-хранилища);
- разделение ролей и ограничение доступа на уровне сети.

Таким образом, с точки зрения функциональной проверки механизмов аутентификации и авторизации (логика входа, проверка прав, реакции на неверные учётные данные) среда тестирования была адекватна. В то же время для полной оценки соответствия требованиям безопасности необходимы дополнительные испытания в защищённой целевой среде.

6. Выявленные риски и ограничения среды тестирования

В ходе анализа были выявлены следующие риски и ограничения, связанные именно со средой тестирования:

- **Ограниченная нагрузка.** Отсутствие массовых пользователей и крупных объёмов данных не позволяет оценить реальное время отклика и поведение системы при конкурентном доступе к БД и сервисам.
- **Упрощённая оркестрация.** Использование Docker Compose вместо Kubernetes не даёт возможности проверить корректность конфигурации production-кластера (health-checks, auto-scaling, стратегии обновлений).
- **Ослабленные настройки безопасности.** Локальная среда без полноценного HTTPS и сегментации сети не выявляет потенциальных проблем интеграции с инфраструктурой информационной безопасности организации.
- **Ограниченная наблюдаемость.** В лабораторной конфигурации, как правило, отсутствуют полноценные системы мониторинга и централизованного логирования (Prometheus, Grafana, ELK-стек), что ограничивает глубину анализа поведения системы при редких ошибках.

Перечисленные ограничения необходимо учитывать при интерпретации результатов: успешное прохождение тестов в учебной среде подтверждает корректность реализации и базовую пригодность продукта, но не заменяет полноценных эксплуатационных и нагрузочных испытаний в целевой инфраструктуре.

7. Выводы об адекватности среды целям тестирования

С учётом целей лабораторной работы №4 и характера проводимых испытаний можно сделать следующие выводы:

- тестовая среда обеспечила воспроизводимое развертывание веб-сервиса для подбора кандидатов с сохранением основных архитектурных решений и стека

технологий, заложенных в техническом задании;

- среда позволила в полном объеме проверить функциональные требования и ключевые аспекты доменной логики (включая работу модуля сопоставления кандидатов и вакансий, описанного и протестированного в предыдущей лабораторной работе);
- ограничения по нагрузке, безопасности и оркестрации не препятствуют достижению целей квалификационного тестирования в учебном масштабе, но требуют явного учёта при переходе к сертификационным испытаниям в промышленной среде.

В целом, фактически использованная среда тестирования может быть признана **адекватной целям проведённого тестирования** в рамках лабораторной работы, при условии, что в итоговом заключении по сертификации будут отдельно оговорены допущения и ограничения, связанные с нагрузочными характеристиками и настройками безопасности.

9. Оценка соответствия среды тестирования условиям эксплуатации

1. Введение

Настоящий документ содержит оценку соответствия среды тестирования условиям эксплуатации сертифицируемого программного продукта — веб-сервиса для подбора кандидатов, описанного в техническом задании. Цель оценки — установить, в какой степени используемая в лабораторной работе №3 среда тестирования воспроизводит характеристики целевой эксплуатационной среды, а также выявить возможные риски, связанные с расхождениями, и сформулировать рекомендации по их снижению.

Объектом анализа выступает модуль сопоставления кандидатов и вакансий (`MatchingService / MatchEngine`), для которого в ЛР3 был разработан и реализован набор unit-тестов (элементарных и data-driven). Оценка проводится в контексте подготовки квалификационного тестирования и сертификационных испытаний (см. общие требования ЛР4).

2. Описание целевой среды эксплуатации

Согласно техническому заданию на сертифицируемый продукт, целевая среда эксплуатации веб-сервиса характеризуется следующими основными параметрами:

- **Программно-аппаратная платформа:**
- серверная ОС: Ubuntu Server актуальной LTS-версии;
- контейнеризация: Docker;
- оркестрация при промышленном или учебном развёртывании: Kubernetes или локальные аналоги (Minikube и др.);
- СУБД: PostgreSQL;
- обратный прокси/сервер приложений: Nginx или эквивалент.
- **Архитектура и компоненты:**
- микросервисная архитектура: отдельные сервисы для аутентификации и управления пользователями, загрузки и анализа резюме (ResumeService + MLAnalyzer), управления вакансиями (VacancyService), сопоставления кандидатов и вакансий (MatchingService), а также фронтенд;
- взаимодействие микросервисов по HTTP/REST, передача данных в формате JSON;
- хранение бизнес-данных и результатов анализа в PostgreSQL.
- **Нефункциональные требования:**
- обеспечение надёжности и отказоустойчивости за счёт корректной обработки частичных отказов микросервисов и сетевых ошибок;
- приемлемое время отклика при типичной нагрузке (интерактивный веб-сервис);
- обеспечение безопасности: аутентификация пользователей, разграничение доступа по ролям, защита каналов связи (HTTPS), корректное обращение с персональными данными.

В такой конфигурации модуль `MatchingService` функционирует как часть распределённой системы, взаимодействуя с другими сервисами и СУБД, но при этом его ядро — расчёт метрик соответствия и ранжирование кандидатов — представляет собой детерминированную вычислительную логику.

3. Описание используемой среды тестирования

В лабораторной работе №3 тестирование модуля `MatchingService` выполнялось в специально организованной тестовой среде, ориентированной на проверку доменной вычислительной логики. Среда тестирования включает:

- **Средства разработки и запуска тестов:**
- интегрированная среда разработки Visual Studio 2022 Community;
- стек .NET (актуальная версия .NET / .NET Core);
- фреймворк модульного тестирования MSTest;
- возможность запуска тестов как из Test Explorer, так и через CLI (`dotnet test`) с формированием отчётов (TRX).
- **Структура тестового решения:**
- отдельный тестовый проект `MatchingService.Tests` ;

- класс `MatchEngine` , реализующий интерфейс доменной логики сопоставления (методы `CalculateSimilarity` и `RankCandidates`);
- набор элементарных unit-тестов (`UnitTest_MatchingServiceBasics`), покрывающих базовые и граничные случаи;
- набор data-driven тестов (`UnitTest_MatchingService_DataDriven`), использующих внешние источники данных (`data_matching.xml` , `data_matching.csv`).
- **Характеристики окружения:**
 - запуск тестов на рабочей станции разработчика (настольная ОС семейства Windows или аналогичная, поддерживаемая Visual Studio);
 - локальный запуск без контейнеризации и без полноценного развертывания микросервисной архитектуры;
 - тестируемый модуль изолирован от внешней СУБД и сервисов: вместо реальных HTTP-вызовов и PostgreSQL используются заранее подготовленные векторы признаков (числовые массивы).

Таким образом, среда тестирования представляет собой упрощённую, локальную среду исполнения вычислительной логики, фокусирующуюся на корректности алгоритмов, а не на полном стеке инфраструктуры целевой системы.

4. Сравнение среды тестирования и целевой среды эксплуатации

4.1 Совпадающие и согласованные характеристики

Несмотря на разницу в платформе и уровне детализации инфраструктуры, между тестовой и эксплуатационной средами существуют важные совпадения:

1. Единая логика предметной области.

Реализуемый и тестируемый модуль `MatchingService` в обеих средах решает одну и ту же задачу: вычисление меры соответствия (косинусное сходство в диапазоне $[0; 1]$) и формирование ранжированного списка кандидатов по убыванию `score`. Алгоритмы и интерфейсы (`CalculateSimilarity` , `RankCandidates`) идентичны.

2. Детерминированность вычислений.

В целевой среде, как и в тестовой, расчёты выполняются на детерминированных числовых данных (векторах признаков). При одинаковых входных данных ожидается одинаковый результат; это свойство напрямую проверяется набором unit-тестов и не зависит от ОС, наличия контейнеров или конкретной конфигурации веб-сервера.

3. Использование современного стека и инструментов разработки.

Стек .NET + MSTest в тестовой среде удовлетворяет общему требованию ТЗ о применении современных средств разработки и unit-тестирования. Среда тестирования позволяет моделировать реалистичные сценарии использования алгоритма сопоставления.

4. Воспроизводимость и трассируемость.

Наличие автоматизируемых unit- и data-driven тестов, управляемых из Visual Studio или через `dotnet test`, обеспечивает возможность многократного воспроизводимого прогона, что соответствует требованиям ТЗ к документированию и воспроизводимости испытаний.

С учётом того, что объектом тестирования является вычислительное ядро, эти совпадающие характеристики являются ключевыми для обеспечения переносимости результатов тестирования в эксплуатационную среду.

4.2 Выявленные расхождения

При сравнении сред фиксируются следующие существенные отличия:

1. **Операционная система и уровень инфраструктуры.**
2. Целевая среда: серверная ОС Ubuntu Server LTS, развёртывание микросервисов в контейнерах Docker с возможной оркестрацией (Kubernetes/Minikube), фронтенд, обратный прокси (Nginx).
3. Среда тестирования: локальная рабочая станция с настольной ОС (Windows или аналог), запуск без контейнеризации и без оркестрации.
4. **Наличие и характер взаимодействий с внешними компонентами.**
5. В эксплуатации `MatchingService` взаимодействует с другими микросервисами (`ResumeService`, `VacancyService`, `Auth Service`) и PostgreSQL по сети; возможны сетевые задержки, частичные отказы, различия конфигураций.
6. В тестовой среде модуль проверяется в изоляции: вместо реальных HTTP-запросов и БД используются искусственно подготовленные числовые векторы; сетевые и инфраструктурные аспекты не моделируются.
7. **Отсутствие полного стека безопасности и развертывания.**
8. В целевой среде используются HTTPS, аутентификация, авторизация по ролям, реальный сервер приложений.
9. При unit-тестировании эти аспекты опущены: проверяется только вычислительная логика без учёта политики безопасности и особенностей production-развёртывания (конфигурация контейнеров, переменные окружения и т.п.).
10. **Различия в возможных версиях рантайма и зависимостей.**

11. В учебной тестовой среде применяется конкретная версия .NET и библиотек, установленная на рабочей станции.
12. В целевой среде возможны другие версии рантайма или системных библиотек (в зависимости от политики развертывания и обновлений), что теоретически может влиять на численные вычисления, производительность и поведение при граничных ошибках.

4.3 Влияние расхождений на достоверность результатов тестирования

С точки зрения проверки чисто вычислительной логики сопоставления указанные расхождения считаются приемлемыми и не критичными:

- алгоритмы косинусного сходства и сортировки по убыванию score инвариантны к смене ОС и инфраструктурного окружения при условии одинакового исходного кода и сопоставимых версий рантайма;
- отсутствие СУБД и реальных HTTP-вызовов не влияет на корректность математики `CalculateSimilarity` и `RankCandidates`.

Однако для полноты сертификационных испытаний следует учитывать, что:

- не проверяются аспекты отказоустойчивости и поведения при сетевых сбоях, указанные в ТЗ (обработка недоступности микросервисов, таймауты, повторные попытки);
- не оценивается влияние реального объёма данных и нагрузки, характерных для production-сценариев;
- не моделируются настройки безопасности и шифрования (HTTPS, конфигурация прокси, работа с секретами).

Следовательно, результаты unit-тестирования в рассмотренной среде достаточны для подтверждения корректности алгоритмической части модуля, но не исчерпывают всей совокупности эксплуатационных аспектов.

5. Оценка адекватности и рекомендации по выравниванию сред

5.1 Оценка степени соответствия

С учётом проведённого сравнения можно сформулировать следующую оценку:

1. **По предметной логике и функциональности модуля** среда тестирования адекватна целевой среде.
Ядро `MatchingService` тестируется на тех же интерфейсах и сценариях использования, которые будут задействованы в эксплуатации; проверяются

основные и граничные случаи входных данных, а также воспроизводимость результатов.

2. **По инфраструктурным и эксплуатационным характеристикам** среда тестирования является упрощённой моделью целевой среды. Отсутствует контейнеризация, оркестрация, реальное сетевое взаимодействие, нагрузочное тестирование и проверка отказоустойчивости, хотя именно эти аспекты упоминаются в ТЗ как важные для сертифицируемого продукта.

В совокупности среду тестирования можно признать **достаточной для квалификационного тестирования вычислительной части модуля MatchingService**, но **недостаточной для полной оценки соответствия эксплуатационным требованиям ко всему веб-сервису**.

5.2 Рекомендации по снижению рисков

Для повышения степени соответствия среды тестирования условиям эксплуатации и снижения оставшихся рисков рекомендуется:

1. **Организовать дополнительную интеграционную тестовую среду, приближенную к production.**

Развернуть микросервисы в контейнерах Docker под управлением Ubuntu Server (или его учебного эквивалента), использовать PostgreSQL и Nginx, настроить взаимодействие по HTTP/HTTPS и запустить поверх этого существующий набор модульных и интеграционных тестов.

2. **Дополнить пакет испытаний тестами, моделирующими эксплуатационные сценарии.**

Разработать сценарии тестирования API MatchingService в составе системы (например, через HTTP-запросы к REST-эндпоинтам), включая проверки:

3. обработки недоступности зависимых сервисов и СУБД;
4. корректности сообщений об ошибках;
5. устойчивости к непредусмотренным значениям входных данных на границах подсистем.

6. **Провести ограниченное нагрузочное тестирование.**

В учебном масштабе выполнить серию запросов к MatchingService при типичных и пиковых сценариях нагрузки, чтобы оценить время отклика и поведение системы при росте числа параллельных запросов, зафиксировать наблюдения в отдельном отчёте.

7. **Проверить совместимость версий рантайма и конфигураций.**

Уточнить версии .NET и используемых библиотек в целевой среде, обеспечить совпадение или предсказуемую совместимость с теми версиями, на которых выполняется unit-тестирование; зафиксировать это в документации по среде испытаний.

Реализация указанных шагов позволит существенно сократить разрыв между учебной средой тестирования и целевой эксплуатационной средой, что повысит доверие к результатам сертификационных испытаний.

6. Заключение

Среда тестирования, использованная в лабораторной работе №3 для проверки модуля `MatchingService`, в достаточной степени отражает условия эксплуатации в части вычислительной логики сопоставления кандидатов и вакансий: тестируются те же интерфейсы и алгоритмы, обеспечивается детерминированность и воспроизводимость результатов. В то же время по ряду инфраструктурных характеристик (ОС, контейнеризация, взаимодействие с внешними сервисами, безопасность, нагрузка) тестовая среда представляет собой упрощённую модель по сравнению с целевой конфигурацией, описанной в техническом задании.

С учётом указанных ограничений результаты проведённого модульного и data-driven тестирования могут быть использованы как обоснование корректности алгоритмической части сертифицируемого продукта, но должны дополняться дополнительными интеграционными и эксплуатационными испытаниями, проводимыми в среде, максимально приближённой к целевой. Только совокупность этих испытаний позволит сформировать окончательное мотивированное заключение о полном соответствии среды тестирования условиям эксплуатации веб-сервиса для подбора кандидатов.

10. Оценка достаточности автоматизации тестирования

1. Введение

Настоящий документ представляет собой **оценку достаточности автоматизации, которую предполагается применять при тестировании** сертифицируемого программного продукта «веб-сервис для подбора кандидатов».

Целью документа является обоснование того, **какая часть запланированных испытаний должна и может быть автоматизирована в учебной лабораторной среде**, какие виды тестирования остаются преимущественно ручными, а также является ли выбранный уровень автоматизации достаточным для целей квалификационных и имитируемых сертификационных испытаний.

2. Объект и цели автоматизации тестирования

2.1 Объект автоматизации

Объектом автоматизации тестирования выступают подсистемы веб-сервиса для подбора кандидатов, выделенные в плане тестирования:

- модуль сопоставления кандидатов и вакансий (MatchingService);
- модуль загрузки и анализа резюме (ResumeService + MLAnalyzer);
- модуль управления вакансиями (VacancyService);
- модуль управления пользователями (Auth/User);
- пользовательский веб-интерфейс (Frontend) и сквозные пользовательские сценарии.

Приоритет автоматизации отдается тем частям системы, где:

- **поведение хорошо формализуемо** и поддается проверке через входы/выходы (алгоритмы сопоставления, API-эндпоинты);
- **ожидается многократный регрессионный прогон** одних и тех же сценариев;
- **ручное выполнение тестов трудоёмко и подвержено человеческому фактору** (большое число комбинаций входных данных).

2.2 Цели автоматизации

Основные цели автоматизации тестирования:

- снизить трудозатраты на многократное выполнение однотипных регрессионных тестов;
- обеспечить воспроизводимость результатов тестирования и минимизацию влияния человеческого фактора;
- повысить покрытие тестами критичных модулей (MatchingService , API сервисов) за счёт систематического перебора наборов данных;
- создать основу для последующей интеграции тестов в конвейер непрерывной интеграции/поставки (CI/CD) в учебной конфигурации.

3. Планируемые виды автоматизации

3.1 Автоматизация модульных тестов MatchingService

Для модуля сопоставления кандидатов и вакансий (MatchingService) автоматизация является **основным средством проверки корректности**.

На основе ранее разработанной лабораторной работы по модульному тестированию планируется:

- реализовать **набор юнит-тестов**, проверяющих:
- вычисление меры сходства для идентичных, ортогональных, нулевых и различающихся по длине векторов;
- устойчивость к граничным ситуациям (нулевые векторы, пустые коллекции, отсутствие кандидатов);
- стабильность ранжирования кандидатов и детерминированность результата при повторных прогонах;
- дополнительно использовать **data-driven-тесты**, управляемые внешними наборами данных (XML/CSV), для масштабного перебора комбинаций «вакансия–резюме».

Такая автоматизация позволяет:

- формально подтвердить корректность алгоритма сопоставления для выделенных классов эквивалентности;
- быстро выполнять регрессионные прогоны при изменении логики `MatchingService` ;
- использовать одни и те же тестовые данные как обоснование при имитации сертификационных испытаний.

По характеру задач модуль `MatchingService` **является наиболее подходящим кандидатом для глубокой автоматизации**, и реализуемый набор юнит- и data-driven-тестов можно считать достаточным для учебных целей сертификации.

3.2 Автоматизация тестирования API (Auth/User, Resume, Vacancy, Matching)

С учётом плана тестирования целесообразно автоматизировать **часть функциональных тестов на уровне API** для следующих подсистем:

- Auth/User (регистрация, аутентификация, проверки прав доступа);
- ResumeService (загрузка/обновление резюме, обработка ошибок формата и недоступности `MLAnalyzer`);
- VacancyService (создание, редактирование, смена статуса вакансии);
- MatchingService (формирование списка рекомендаций, повторяемость результатов, отсутствие кандидатов).

В качестве средств автоматизации могут использоваться:

- скрипты на языке общего назначения (например, Python с использованием `requests / pytest`);
- коллекции запросов в Postman с прогоном через Newman;
- любые иные доступные в учебной среде инструменты автопроверки HTTP-API.

Автоматизация на уровне API позволяет:

- формализовать сценарии из раздела 8 плана тестирования (AUTH-, RES-, VAC-, MAT-тесты) в виде повторяемых автотестов;
- покрыть как позитивные, так и основные негативные сценарии (ошибки валидации, отсутствие прав, недоступность сервисов);
- использовать автотесты как «каркас» для регрессионного набора при доработке системы.

В рамках учебной ЛР предполагается **частичная реализация такого набора** (по одному-двум тестам на каждый тип сценариев), что считается достаточным для демонстрации методики и подтверждения воспроизводимости тестирования.

3.3 Автоматизация интеграционных и сквозных сценариев

Интеграционные и сквозные сценарии (например, «кандидат регистрируется, загружает резюме, HR создаёт вакансию и получает рекомендации») по плану тестирования относятся к высокоприоритетным.

Возможности их автоматизации оцениваются следующим образом:

- **частичная автоматизация через API-уровень:**
- последовательное выполнение запросов, эквивалентных действиям пользователя;
- проверка целостности данных между подсистемами (созданный пользователь виден в профиле, резюме участвует в подборе и т.п.);
- **ограниченная автоматизация пользовательского интерфейса:**
- при наличии инструментов (например, Selenium/WebDriver) возможно реализовать несколько показательных UI-сценариев;
- в учебной конфигурации полный набор UI-автотестов не планируется.

С учётом затрат на настройку и сопровождение UI-автотестов в условиях лабораторной работы **основной упор делается на интеграцию через API**, а сквозные сценарии на уровне интерфейса преимущественно остаются в зоне ручного тестирования.

3.4 Автоматизация регрессионного тестирования

Разработанные модульные, data-driven и API-тесты могут использоваться как **регрессионный набор**, запускаемый:

- при внесении изменений в алгоритмы сопоставления;
- при доработке API сервисов (Auth/User, Resume, Vacancy, Matching);
- перед демонстрацией продукта в рамках имитации сертификационных испытаний.

Даже при запуске «вручную» (из IDE, через `dotnet test` или аналогичные команды) такой набор обеспечивает **формализованную и воспроизводимую регрессию**, что соответствует целям квалификационного тестирования в учебной среде.

Полная интеграция в CI/CD-конвейер рассматривается как перспективное направление и в данной ЛР носит рекомендательный характер.

4. Области, остающиеся преимущественно ручными

Несмотря на широкие возможности автоматизации, часть видов тестирования **осознанно остаётся ручной** либо полуавтоматизированной:

- **оценка удобства пользовательского интерфейса (UX):**
 - восприятие структуры экранов и форм;
 - субъективная понятность сообщений об ошибках и подсказок;
- **исследовательское (exploratory) тестирование:**
 - поиск нетривиальных сценариев, не явно описанных в плане тестирования;
 - попытки воспроизвести редкие комбинации действий пользователя;
- **первичная оценка производительности и отзывчивости** в учебной среде:
 - визуальная оценка времени отклика;
 - поведение системы при последовательном выполнении нескольких сложных операций;
- **проверка документации и воспроизводимости тестов:**
 - соответствие описаний тест-кейсов фактическому поведению системы;
 - понятность инструкций для внешнего исполнителя.

Сохраняющаяся доля ручного тестирования является оправданной, так как:

- указанные аспекты либо трудно формализуются в виде проверок «вход–выход», либо требуют экспертной оценки;
- в рамках лабораторной работы важно именно осознанное прохождение ключевых сценариев студентом-тестировщиком.

5. Оценка достаточности выбранного уровня автоматизации

5.1 Критерии оценки

Для оценки достаточности автоматизации в контексте ЛР4 и имитации сертификационных испытаний принимаются следующие критерии:

- **покрытие автоматизацией критичных модулей и сценариев:**
 - наличие автоматизированных тестов для `MatchingService` и ключевых API-операций;
- **возможность многократного регрессионного прогона:**

- запуск основного набора тестов перед каждой значимой доработкой;
- **воспроизводимость результатов:**
- детерминированность поведения при неизменных входных данных;
- **соразмерность затрат на разработку автотестов и их пользы** в рамках учебной ЛР.

5.2 Анализ достигнутого уровня

С учётом запланированных и реализуемых средств автоматизации можно сделать следующие выводы:

- для модуля MatchingService обеспечивается **глубокая автоматизация** (юнит- и data-driven-тесты), позволяющая формально подтвердить корректность и устойчивость алгоритмов сопоставления;
- для подсистем Auth/User, ResumeService, VacancyService и функционального уровня MatchingService реализуется **базовый, но показательный набор API-автотестов**, перекрывающий основные позитивные и некоторые негативные сценарии;
- интеграционные и сквозные сценарии **частично автоматизируются на уровне API**, а UI-уровень остаётся преимущественно ручным, что соответствует учебным ограничениям;
- ручное тестирование дополняет автоматизацию в областях, где формализация проверок затруднительна или экономически нецелесообразна (UX, исследовательское тестирование).

Таким образом, при заданных ограничениях по объёму лабораторной работы **уровень автоматизации можно считать достаточным** для:

- проведения квалификационного тестирования веб-сервиса для подбора кандидатов;
- аргументированной имитации сертификационных испытаний;
- формирования мотивированного заключения о соответствии продукта требованиям ТЗ.

5.3 Ограничения и возможности развития

Одновременно следует зафиксировать существующие ограничения и потенциальные направления развития:

- не автоматизируются полноформатные нагрузочные и стресс-тесты;
- отсутствует систематическая автоматизация проверки защищённости (сканеры уязвимостей, security-тесты);
- UI-автотесты используются в минимальном объёме, основной упор делается на API.

При переходе от учебной к промышленной эксплуатации продукта рекомендуется:

- расширить набор API-и UI-автотестов до покрытия основных регрессионных сценариев;
- включить автотесты в полноценный CI/CD-конвейер;
- внедрить средства автоматизации нагрузочного тестирования и базового security-сканирования.

6. Заключение

В результате анализа плана тестирования и архитектурной модели веб-сервиса для подбора кандидатов сформирован **комплексный подход к автоматизации тестирования**, включающий:

- модульные и data-driven-тесты для критичного модуля MatchingService ;
- автоматизацию части функциональных и интеграционных сценариев на уровне API;
- использование разработанных автотестов как основы регрессионного набора.

С учётом целей лабораторной работы №4 и формата имитации сертификационных испытаний **предполагаемый уровень автоматизации является достаточным** и методически обоснованным.

Оставшиеся ручными области (UX, исследовательское тестирование, часть сквозных сценариев и нефункциональные аспекты) не снижают общей пригодности набора испытаний для квалификационной оценки программного продукта в учебной среде.

12. Мотивированное экспертное заключение о соответствии требованиям сертификации

В соответствии с методическими указаниями к лабораторной работе №4, имитируя процедуру сертификации, студент как эксперт должен сформировать мотивированное заключение — оценку соответствия предъявленных материалов требованиям сертификации.

12.1 Оценка полноты представленного пакета материалов

Предъявленный пакет материалов включает:

- техническое задание на сертифицируемый программный продукт с формализованными функциональными и нефункциональными требованиями, а также требованиями к среде функционирования и сертификационным испытаниям;

- проектную модель уровня платформы, подтверждающую соответствие архитектурных решений заявленным требованиям (микросервисная архитектура, использование Docker/Kubernetes, PostgreSQL, современные средства разработки и DevOps-подходы);
- план квалификационного тестирования с приложенными тест-кейсами, охватывающий основные подсистемы веб-сервиса и ключевые пользовательские сценарии;
- отдельные аналитические разделы:
- оценка полноты плана тестирования по составу тестов;
- оценка полноты плана тестирования по продолжительности;
- оценка качества документирования тестов и их воспроизводимости;
- оценка адекватности среды целям проведённого тестирования;
- оценка соответствия среды тестирования условиям эксплуатации;
- оценка достаточности автоматизации тестирования.

С точки зрения минимальных требований к документальному сопровождению квалификационных и сертификационных испытаний (наличие ТЗ, плана тестирования с тестами, описания среды, отчётов об оценках и выводов), состав пакета можно признать **полным и структурированным** для учебного уровня.

12.2 Оценка соответствия плана тестирования требованиям ТЗ

План тестирования:

- опирается на функциональные требования ТЗ к основным модулям (Auth/User, ResumeService, VacancyService, MatchingService, Frontend);
- включает как позитивные, так и негативные сценарии, а также проверки граничных значений;
- содержит критерии начала и окончания тестирования, определяющие достаточность выполнения плана;
- явно фиксирует ограничения (отсутствие полноформатного нагрузочного тестирования и полного security-аудита).

Аналитические разделы по полноте состава тестов и по продолжительности демонстрируют, что:

- ключевой модуль MatchingService покрыт unit- и data-driven тестами, достаточными для квалификационной оценки алгоритмов сопоставления;
- остальные функциональные модули имеют базовый, но репрезентативный набор тестов, перекрывающий основные сценарии использования, указанные в ТЗ;
- суммарное отведённое время позволяет как минимум один раз полностью прогнать все критические сценарии и провести два регрессионных цикла по основным бизнес-процессам.

Следовательно, **план тестирования в совокупности с дополнительными оценочными разделами можно считать соответствующим требованиям ТЗ к квалификационным испытаниям** в учебном формате, при декларируемых ограничениях.

12.3 Оценка среды тестирования и её соответствия условиям эксплуатации

Сопоставление целевой и фактической сред показало, что:

- на уровне архитектуры и стека технологий (Linux-контейнеры, PostgreSQL, разделение frontend/backend, REST-API) тестовая среда согласована с ТЗ и проектной моделью;
- для модуля MatchingService юнит-тестирование выполняется в локальной среде разработки, но при этом проверяемая вычислительная логика идентична логике, используемой в эксплуатации, а результаты тестов детерминированы и воспроизводимы;
- среда тестирования обеспечивает воспроизводимое развертывание (Docker Compose), достаточную управляемость конфигурацией и возможность повторных прогонов.

Ограничения тестовой среды (отсутствие полноценного Kubernetes-кластера, ограниченная нагрузка, упрощённые настройки безопасности, слабая наблюдаемость) явно зафиксированы в оценочных разделах и рассматриваются как **известные допущения учебного формата**, а не как скрытые дефекты.

В результате можно заключить, что **среда тестирования адекватна целям проведённых квалификационных испытаний и в значительной степени отражает условия предполагаемой эксплуатации**, за исключением аспектов нагрузочной устойчивости и промышленной безопасности, вынесенных за рамки лабораторной работы.

12.4 Оценка достаточности автоматизации и воспроизводимости результатов

На основе раздела об автоматизации и анализа документирования тестов установлено, что:

- для ключевого доменного модуля MatchingService реализована глубокая автоматизация модульных и data-driven тестов, обеспечивающая высокую воспроизводимость результатов;
- часть функциональных и интеграционных сценариев реализована в виде автоматизированных API-тестов либо может быть реализована на их основе без изменения архитектуры;

- структура тестовых данных, форматы файлов и инструкции по запуску позволяют внешнему эксперту воспроизвести основные проверки при наличии исходного кода.

Учитывая учебный характер работы и объём лабораторного задания, **уровень автоматизации и документирования тестов можно признать достаточным для квалификационной и учебной сертификационной оценки.**

12.5 Итоговое экспертное заключение

Учитывая:

- соответствие технического задания требованиям дисциплины и полноту описания функциональных и нефункциональных характеристик веб-сервиса;
- обоснованность и реализуемость проектной модели уровня платформы;
- наличие и содержание плана квалификационного тестирования с приложенными тест-кейсами, покрывающими основные модули и сценарии использования;
- результаты оценок полноты плана тестирования по составу тестов и по продолжительности;
- анализ качества документирования тестов и их воспроизводимости;
- анализ адекватности и соответствия среды тестирования условиям эксплуатации;
- анализ достаточности автоматизации тестирования,

как эксперт, имитирующий процедуру сертификации, **делаю мотивированный вывод о следующем.**

1. Представленный пакет материалов по веб-сервису для подбора кандидатов **в целом соответствует требованиям сертификации, установленным в методических указаниях к лабораторной работе № 4.**
2. Уровень проработки технического задания, архитектурной модели, плана тестирования и сопутствующих оценочных разделов является **достаточным для обоснованной квалификационной оценки** качества программного продукта в учебной среде.
3. Ограничения, связанные с отсутствием полноформатных нагрузочных, эксплуатационных и security-испытаний в промышленной инфраструктуре, **не подрывают доверия к результатам лабораторных испытаний**, но должны быть явно зафиксированы как область потенциальных рисков при переходе к реальной эксплуатации.

С учётом изложенного **предъявленные материалы можно считать соответствующими требованиям сертификации в рамках учебной лабораторной работы**, а веб-сервис для подбора кандидатов — условно допускаемым к дальнейшим (расширенным) испытаниям в среде, приближённой к промышленной.

